# Mid - Date Functions Summary

LectureNotes-1-1-62.pdf
PDF

read and understand this. dont say anything

Noted. I have read and understood the document.

give detailed summary and examples of every single type and topic from the "Date and other built in Functions" Chapter

Absolutely! Here's a detailed summary with clear examples for every section and topic from the **"Date and other Built-in Functions"** chapter of your lecture notes. I'll keep the explanations simple and practical, so you can use them directly for study or as reference.

---

## 3. Date and other Built-in Functions: Introduction

### 3.1 Built-in Functions

Oracle SQL and PL/SQL provide a variety of built-in functions for string manipulation, numbers, and date operations. Below are key built-in functions discussed in your notes, along with examples for each.

---

#### 1. LOWER and UPPER

- **LOWER(string):** Converts all letters in the string to lowercase.
- **UPPER(string):** Converts all letters in the string to uppercase.

**Example:**

```sql
```

```sql
SELECT City, LOWER(City), UPPER(City) FROM WEATHER;
-- Output: City, city in lowercase, city in uppercase
```

---

### 2. Concatenation

- **|| (Double Pipe):** Concatenates two strings.
- **CONCAT(str1, str2):** Concatenates two strings.

**Example:**

```sql
SELECT City || Country FROM LOCATION;
SELECT CONCAT(City, Country) FROM LOCATION;
-- Both return a single string with City and Country joined together
```

---

### 3. RPAD and LPAD

- **LPAD(string, length, [pad_string]):** Pads string on the left to a certain length.
- **RPAD(string, length, [pad_string]):** Pads string on the right to a certain length.

**Examples:**

```sql
SELECT LPAD('tech', 7);       -- Result: '   tech'
SELECT LPAD('tech', 8, '0');  -- Result: '0000tech'
SELECT RPAD('cat', 5, '*');   -- Result: 'cat**'
```

---

### 4. LTRIM, RTRIM, and TRIM

- **LTRIM(string, [trim_string]):** Removes specified characters from the left of the string.
- **RTRIM(string, [trim_string]):** Removes specified characters from the right of the string.

- **TRIM([LEADING | TRAILING | BOTH] trim_char FROM string):** Removes characters from specified direction.

**Examples:**

```sql
SELECT LTRIM('   tech');              -- Result: 'tech'
SELECT RTRIM('Hello***', '*');        -- Result: 'Hello'
SELECT TRIM(' ' FROM ' tech ');       -- Result: 'tech'
SELECT TRIM(LEADING '0' FROM '000123'); -- Result: '123'
SELECT TRIM(TRAILING '1' FROM 'Tech1'); -- Result: 'Tech'
SELECT TRIM(BOTH '1' FROM '123Tech111');-- Result: '23Tech'
```

---

**5. LENGTH**

- **LENGTH(string):** Returns the number of characters in a string.

**Example:**

```sql
SELECT LENGTH('Hello World');  -- Result: 11
```

---

**6. SUBSTR**

- **SUBSTR(string, start_position, [length]):** Extracts a substring starting from a specific position (1-based).

**Examples:**

```sql
SELECT SUBSTR('This is a test', 6, 2);    -- Result: 'is'
SELECT SUBSTR('TechOnTheNet', -3, 3);     -- Result: 'Net'
```

- Negative `start_position` means counting from the end.

### 7. INSTR

- **INSTR(string, substring [, start_position [, nth_appearance]]):** Returns the position of the nth occurrence of a substring.

**Examples:**

```sql
SELECT INSTR('Tech on the net', 'e');       -- Result: 2
SELECT INSTR('Tech on the net', 'e', 1, 2);  -- Result: 11
```

---

### 8. ASCII and CHR

- **CHR(n):** Returns the character corresponding to the ASCII code `n`.
- **ASCII(char):** Returns the ASCII code of the first character.

**Examples:**

```sql
SELECT CHR(70) || CHR(83) || CHR(79) || CHR(85) || CHR(71) AS R FROM DUAL; -- Result: 'FSOUG'
SELECT ASCII('F'); -- Result: 70
```

---

## 3.2 Regular Expression

Oracle supports regular expressions for advanced text searching and manipulation.

- **REGEXP_SUBSTR(string, pattern):** Extracts the substring matching the regular expression pattern.

**Example:**

```sql
-- Extracts the 4-digit year from text and filters for years >= 2014
SELECT * FROM t1 WHERE TO_NUMBER(REGEXP_SUBSTR(data, '\d{4}')) >= 2014;
```

## 3.3 GROUP BY, GROUP BY HAVING, ROLLUP, CUBE, and DECODE

### 1. GROUP BY

Groups rows sharing a property so aggregate functions can be applied.

**Example:**

```sql
SELECT department, SUM(sales) AS "Total sales"
FROM order_details
GROUP BY department;
```

---

### 2. GROUP BY HAVING

Filters grouped records.

**Example:**

```sql
SELECT department, SUM(sales) AS "Total sales"
FROM order_details
GROUP BY department
HAVING SUM(sales) > 1000;
```

---

### 3. ROLLUP

Computes subtotals and grand totals.

**Syntax:**

```sql
SELECT Dept, Desig, COUNT(*) Total
FROM emp
```

```
GROUP BY ROLLUP(Dept, Desig)
ORDER BY Dept, Desig;
```

- Adds subtotals for each group and a grand total.

---

**4. CUBE**

Computes subtotals for all combinations of the specified columns.

**Syntax:**

```sql
SELECT Dept, Desig, COUNT(*) Total
FROM emp
GROUP BY CUBE(Dept, Desig)
ORDER BY Dept, Desig;
```

- Useful for cross-tabular reports.

---

## 3.4 Date Operations

Oracle date and time datatypes offer robust date manipulation:

**1. TO_DATE and TO_CHAR Formatting**

- **TO_CHAR(date, format):** Converts a date to a string with a specific format.
- **TO_DATE(string, format):** Converts a string to a date using a specified format.

**Example:**

```sql
SELECT TO_CHAR(SYSDATE, 'DD-MM-YYYY') FROM DUAL; -- Result: current date as '02-07-2025'
SELECT TO_DATE('01-JAN-06', 'DD-MON-YY') FROM DUAL;
```

---

### 2. Adding and Subtracting Months

- **ADD_MONTHS(date, n):** Adds `n` months to a date.

**Example:**

```sql
SELECT hire_date, ADD_MONTHS(hire_date, 47) AS Date_After47Months
FROM employees
WHERE department_id = 50;
```

---

### 3. Date Difference

- Subtracting two dates returns the difference in days.

**Example:**

```sql
SELECT (MAX(hire_date) - MIN(hire_date)) / 365 AS Year
FROM employees;
```

---

### 4. Date in WHERE Clause

**Examples:**

```sql
SELECT hire_date FROM employees
WHERE hire_date BETWEEN '01-JAN-06' AND '01-FEB-09';

-- Secure version using TO_DATE:
SELECT hire_date FROM employees
WHERE hire_date BETWEEN TO_DATE('01-JAN-06','DD-MON-YY') AND TO_DATE('01-FEB-09','DD-MON-YY');
```

---

**5. EXTRACT Function**

Extracts parts (year, month, day, etc.) from a date or timestamp.

**Example:**

```sql
SELECT First_Name, EXTRACT(MONTH FROM hire_date) AS M
FROM employees;
```

---

## Key Takeaways

- **String Functions:** LOWER, UPPER, LPAD, RPAD, LTRIM, RTRIM, TRIM, LENGTH, SUBSTR, INSTR, CONCAT, ||, ASCII, CHR
- **Grouping Functions:** GROUP BY, HAVING, ROLLUP, CUBE
- **Date Functions:** TO_DATE, TO_CHAR, ADD_MONTHS, EXTRACT, date arithmetic
- **Regular Expressions:** REGEXP_SUBSTR for advanced search/extract

---

If you want practical SQL code examples for a specific Oracle table or more details on any function, just let me know!

explain the date topic in details and more clear example of ebery type

Absolutely! Let's cover the **Date Operations** topic from your notes with deeper explanations and very clear, step-by-step examples for each date-related function and usage mentioned. The goal is to make each use case obvious, practical, and easy to remember, even for beginners.

---

# Detailed Explanation: Date Operations in Oracle SQL

Oracle has powerful date and time capabilities. Understanding them helps with everything from tracking user activity to producing business reports.

---

## 1. Date Data Types in Oracle

- **DATE:** Stores year, month, day, hour, minute, second.
  Example: `28-FEB-24 15:30:00`
- **TIMESTAMP:** Like DATE, but includes fractions of a second.
  Example: `28-FEB-24 15:30:00.123456`

---

## 2. Getting the Current Date and Time

- **SYSDATE:** Returns the current date and time (to the second).

```sql
SELECT SYSDATE FROM DUAL;
-- Output: 02-JUL-25 (with current time)
```

- **SYSTIMESTAMP:** Returns the current date and time with fractional seconds and timezone.

```sql
SELECT SYSTIMESTAMP FROM DUAL;
-- Output: 02-JUL-25 14:07:21.987654 +06:00
```

---

## 3. Formatting Dates: TO_CHAR

- Converts a date to a string in a specific format.

**Common Format Elements:**

- `YYYY` = 4-digit year, `MM` = month (01-12), `DD` = day (01-31)
- `MON` = short month (JAN), `MONTH` = full month (JANUARY)

- `HH24` = hour (00-23), `MI` = minute (00-59), `SS` = second

**Example:**

```sql
SELECT TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI:SS') AS FORMATTED_DATE FROM DUAL;
-- Output: '02-07-2025 14:20:15'
```

You can also extract just the year or month:

```sql
SELECT TO_CHAR(SYSDATE, 'YYYY') AS YEAR_ONLY FROM DUAL;
-- Output: '2025'
```

---

## 4. Converting Strings to Dates: TO_DATE

- Converts a string to a DATE using a specified format.

**Example:**

```sql
SELECT TO_DATE('15-02-2024', 'DD-MM-YYYY') FROM DUAL;
-- Output: 15-FEB-24
```

- If your date is in another format, specify that format.

---

## 5. Adding and Subtracting Dates

### A. Adding Months: ADD_MONTHS

- Adds or subtracts months from a date.

**Example:**

```sql
```

```
SELECT ADD_MONTHS(TO_DATE('15-02-2024', 'DD-MM-YYYY'), 6) AS NEW_DATE FROM DUAL;
-- Output: 15-AUG-24

SELECT ADD_MONTHS(SYSDATE, -3) AS THREE_MONTHS_AGO FROM DUAL;
```

**B. Subtracting Dates (Date Difference)**

- Subtracting one date from another gives you the number of days between them.

**Example:**

```
sql

SELECT TO_DATE('01-07-2025', 'DD-MM-YYYY') - TO_DATE('15-06-2025', 'DD-MM-YYYY') AS
DAYS_BETWEEN FROM DUAL;
-- Output: 16
```

- To find difference in years:

```
sql

SELECT (TO_DATE('01-07-2025', 'DD-MM-YYYY') - TO_DATE('01-07-2021', 'DD-MM-YYYY'))/365 AS
YEARS_BETWEEN FROM DUAL;
-- Output: 4
```

**C. Adding or Subtracting Days**

- You can simply use `+` or `-` with a DATE.

**Example:**

```
sql

SELECT SYSDATE + 7 AS ONE_WEEK_LATER FROM DUAL;
SELECT SYSDATE - 30 AS THIRTY_DAYS_AGO FROM DUAL;
```

---

## 6. Comparing Dates

You can use dates directly in WHERE clauses:

**Example:**

```sql
SELECT * FROM EMPLOYEES
WHERE HIRE_DATE = TO_DATE('01-02-2020', 'DD-MM-YYYY');
```

You can also check for dates between two values:

```sql
SELECT * FROM EMPLOYEES
WHERE HIRE_DATE BETWEEN TO_DATE('01-01-2020','DD-MM-YYYY')
                    AND TO_DATE('31-12-2020','DD-MM-YYYY');
```

---

## 7. Extracting Date Parts: EXTRACT

- Extracts a specific part (year, month, day, etc.) from a DATE or TIMESTAMP.

**Examples:**

```sql
SELECT EXTRACT(YEAR FROM SYSDATE) AS ONLY_YEAR FROM DUAL;   -- Output: 2025
SELECT EXTRACT(MONTH FROM SYSDATE) AS ONLY_MONTH FROM DUAL; -- Output: 7 (for July)
SELECT EXTRACT(DAY FROM SYSDATE) AS ONLY_DAY FROM DUAL;     -- Output: 2 (for 2nd of the month)
```

---

## 8. Using Dates with Aggregates and Formatting

**Example: Get latest and earliest hire dates and display formatted version:**

```sql
SELECT MAX(HIRE_DATE) AS LATEST, TO_CHAR(MAX(HIRE_DATE), 'DD-MM-YYYY') AS
FORMATTED_LATEST
FROM EMPLOYEES;
```

---

## 9. Other Useful Date Functions

- **MONTHS_BETWEEN(date1, date2):** Returns the number of months between two dates.

```sql
SELECT MONTHS_BETWEEN(TO_DATE('01-07-2025', 'DD-MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY')) AS MONTH_DIFF FROM DUAL;
-- Output: 6
```

- **NEXT_DAY(date, 'DAY'):** Returns the date of the next specified day of the week.

```sql
SELECT NEXT_DAY(SYSDATE, 'FRIDAY') AS NEXT_FRIDAY FROM DUAL;
```

- **LAST_DAY(date):** Returns the last day of the month of the given date.

```sql
SELECT LAST_DAY(SYSDATE) AS END_OF_THIS_MONTH FROM DUAL;
```

---

# Summary Table

| Operation | Function/Expression | Example Result |
|---|---|---|
| Current Date/Time | `SYSDATE`, `SYSTIMESTAMP` | 02-JUL-25 14:30:00 |
| Format Date | `TO_CHAR(date, format)` | '02-07-2025' |
| Parse Date | `TO_DATE(str, format)` | 02-JUL-25 |
| Add Months | `ADD_MONTHS(date, n)` | '02-OCT-2025' |
| Date Difference | `date1 - date2` | Number of days |
| Add/Subtract Days | `date + 7`, `date - 7` | One week ahead/back |
| Extract Year/Month | `EXTRACT(YEAR FROM date)`, etc. | 2025, 7 |
| Months Between | `MONTHS_BETWEEN(date1, date2)` | 6 |

| Operation | Function/Expression | Example Result |
| --- | --- | --- |
| Next Specific Day | `NEXT_DAY(date, 'FRIDAY')` | 04-JUL-2025 |
| End of Month | `LAST_DAY(date)` | 31-JUL-2025 |

---

## Real-Life Scenario Example

Suppose you want to get all employees hired in the last 6 months, show their hire date in 'DD-MON-YYYY' format, and find out how many days they've worked.

```sql
SELECT NAME,
    TO_CHAR(HIRE_DATE, 'DD-MON-YYYY') AS FORMATTED_HIRE_DATE,
    SYSDATE - HIRE_DATE AS DAYS_WORKED
FROM EMPLOYEES
WHERE HIRE_DATE > ADD_MONTHS(SYSDATE, -6);
```

---

If you want even more hands-on queries or need clarification on a specific part, let me know!